



The Unified Cloud Binary: Collapsing a Microservices Control Plane into One Go Process

Zach Kelling

Hanzo AI

HIP-0106 Revision 1

The Unified Cloud Binary: Collapsing a Microservices Control Plane into One Go Process

Zach Kelling*

Hanzo Industries Inc (Techstars '17)

Los Angeles, California

<https://hanzo.ai>

HIP-0106 Revision 1

Abstract

We describe the Hanzo *unified cloud binary* (`hanzoai/cloud`, HIP-0106): a single statically linked Go executable that mounts every platform subsystem—identity, key management, the Base application runtime, the API gateway, model inference, commerce, a virtual file system, message queues, DNS, and observability—into one operating-system process. The web substrate is `hanzoai/zip`, a framework built on Fiber v3 that embeds (i) `goja`, a pure-Go ECMAScript engine, for in-process evaluation of JavaScript handlers; (ii) the `esbuild` Go API for transpiling TypeScript and modern JavaScript to ES5 at build or load time; and (iii) ZAP, a capability RPC with promise pipelining, exposed over HTTP and WebSocket to external clients and reused as the in-process dispatch seam through a canonical `zaprpc.Registry`. The architecture lets an existing TypeScript backend port in unchanged by transpiling to ES5 and running inside the embedded engine, then be rewritten as native Go in place with no change to the HTTP or RPC contract. Within the process, calls between subsystems are Go function invocations rather than network round-trips, eliminating the service mesh, sidecars, per-call mutual TLS, and the container-per-service tax. The same artifact serves `api.hanzo.ai`, `api.lux.cloud`, `api.zoo.cloud`, and every white-label tenant; brand and the set of enabled subsystems are deployment configuration. We give the architecture, a migration path validated against an in-flight tRPC-to-ZAP port, a sharp boundary between in-process evaluation and the separate sandboxed `hanzoai/runtime`, and a cost model grounded in published service-mesh and microservices-overhead measurements.

1 Introduction

A control plane for a multi-tenant SaaS platform of this shape—identity, keys, an application runtime, a gateway, inference, commerce, storage, messaging—is conventionally decomposed into a constellation of independently deployed services. Each service runs in its own container, is fronted by a sidecar proxy, communicates with its peers over the network, and is wired together by a service mesh that terminates mutual TLS and enforces policy on every hop. This decomposition is the default not because the workload demands

it but because the tooling makes it the path of least resistance [8].

For a platform whose subsystems are co-released, co-owned, and almost always co-located on the request path, this default is a poor fit. A login that touches identity, then keys, then the gateway pays three network round-trips, three serialization boundaries, and three sidecar traversals to accomplish what is, logically, three function calls. The compute spent on proxies and (de)serialization is pure overhead; the latency is a staircase that grows with the number of hops; and the deploy graph couples teams who must

coordinate version skew across a distributed system they did not need to distribute.

This paper describes the alternative Hanzo has converged on: a single Go binary that mounts every subsystem into one process. We make four claims. First, for co-located subsystems the unified binary removes the dominant sources of microservices overhead—inter-service serialization, the sidecar data path, and per-call mutual TLS—by turning network RPC into function calls (§3, §8). Second, the framework beneath the binary, `zip`, admits existing TypeScript backends without a rewrite by transpiling them to ES5 and evaluating them in an embedded pure-Go engine, then permits an incremental, contract-preserving rewrite to native Go (§4). Third, the same artifact serves many brands and tenants because brand and the enabled subsystem set are configuration, not code (§5). Fourth, there is a principled line between in-process evaluation of *trusted* code and the separate sandboxed runtime for *untrusted* user code (§6). We close with a cost model (§7) anchored in published measurements rather than synthetic numbers.

2 Motivation

Compute spent on plumbing, not work.

A service mesh inserts a proxy on both ends of every call. Measured overheads are not negligible: a systematic study of Istio reports added per-request latency on the order of single-digit milliseconds and substantial CPU consumption attributable to the proxy data path, dominated by connection handling, serialization, and policy evaluation [15]. When the logical operation is a function call’s worth of work, a millisecond of proxy latency and a proxy-sized CPU slice per hop is the bulk of the cost.

Latency is a staircase in the hop count.

Microservice request fan-out turns a single user action into a tree of internal calls. The Death-StarBench study shows that end-to-end tail latency in microservice topologies is governed by the depth and width of this call tree, and that queuing and serialization at each tier compound [5]. (§6).

Each additional network boundary on the critical path adds a serialization round trip and a scheduling boundary; none of that work advances the computation.

The distributed-systems tax is involuntary.

Splitting an in-process call into a network call replaces a deterministic function invocation with something that can partially fail, time out, or reorder—the classic objection that RPC papers over a fundamental semantic gap between local and remote calls [13]. A platform that distributes subsystems it could have co-located pays this tax—retry logic, idempotency keys, circuit breakers, distributed tracing to reconstruct a call tree—without the scaling benefit that would justify it.

Deploy coupling without isolation benefit.

Co-released subsystems gain little operational isolation from separate deployment: a breaking change in the identity contract still requires the gateway to be updated in lockstep. What separate deployment does add is version-skew surface, N pipelines, and N rollouts to coordinate. Amazon Prime Video’s widely cited account of consolidating a distributed media-monitoring pipeline back into a single process—reporting roughly an order-of-magnitude cost reduction for that workload—is a concrete data point that for some shapes the monolith is the correct engineering choice [10]. We do not claim this generalizes to all systems; we claim it applies to a co-located control plane, and we make the conditions explicit.

What we are *not* claiming. The unified binary is not a rejection of horizontal scaling: the process is stateless at the request layer and runs as many replicas as load demands, one per region or per cell. The claim is narrower and sharper: *subsystems that sit on the same request path and ship together should share an address space*. Genuinely independent, separately scaled, or untrusted components stay out of the binary—most importantly the sandboxed code-execution runtime

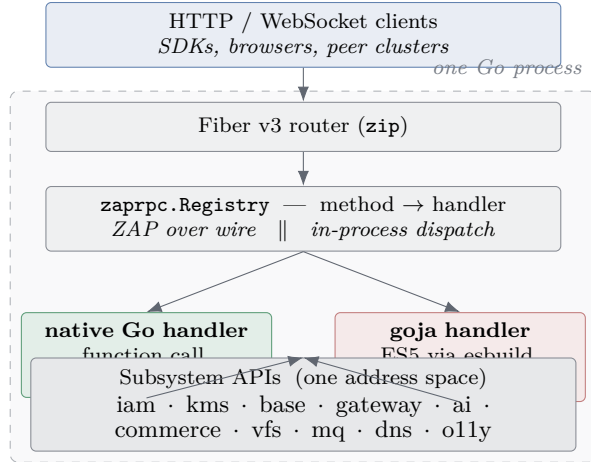


Figure 1: The unified binary request path. Clients reach a Fiber v3 router; `zaprpc.Registry` resolves each method to either a native Go handler or a goja-evaluated handler transpiled to ES5 by esbuild. Both terminate in the same subsystem APIs within one address space; the choice is invisible to clients and peers.

3 Architecture

3.1 The layered binary

Figure 1 shows the request path. HTTP and WebSocket clients reach a Fiber v3 router [3]. The router dispatches to `zip` handlers. A handler resolves to one of two execution strategies: a *native Go* subsystem call, or evaluation of a *transpiled JavaScript* handler inside the embedded `goja` engine [9]. Both strategies terminate in the same subsystem APIs; the choice is invisible to the client and to peer subsystems.

`zip` is a Go module (github.com/hanzoai/zip) that wraps Fiber with the seams the platform needs: a module system (`module.go`) for mounting subsystems, lifecycle and shutdown ordering (`closers.go`), an OpenAPI surface (`openapi.go`) generated from the same specs that drive the `hanzoai/openapi` registry, and—centrally—`zaprpc.Registry`, the one place where a method name resolves to a handler. The registry is the dispatch seam shared by the in-process and over-the-wire paths.

3.2 ZAP: one RPC, two domains

ZAP is a capability RPC modeled on Cap’n Proto’s promise pipelining [11]: a call may be issued against the *result* of an earlier, still-pending call, so a chain of dependent operations collapses into a single round trip rather than one per step. ZAP serves two domains from one definition.

Over the wire, external clients speak ZAP over HTTP (request/response) or WebSocket (bidirectional, for streaming and pipelined chains). This is the public contract: an SDK, a browser, or a peer cluster sees a stable method surface.

Inside the process, the same `zaprpc.Registry` resolves a method name directly to a Go function. A subsystem that needs identity does not open a socket; it calls into the identity module through the registry, and the call is an ordinary function invocation with no serialization. The registry is the decomplexing seam: *policy* (which method is exposed, to whom) is separated from *transport* (wire vs. in-process), and both are separated from *implementation* (Go function vs. goja handler).

3.3 esbuild as the build-time and load-time transpiler

TypeScript and modern JavaScript do not run in `goja` directly; `goja` targets ES5.1 with select later features. `zip` embeds the `esbuild` Go API [14] to transpile and bundle handler sources to ES5. `esbuild` runs in-process (it is itself a Go program), so transpilation needs no external toolchain and no CGO. Two modes are supported: *build-time*, where handlers are transpiled once and embedded into the binary via `go:embed`; and *load-time*, where a handler bundle is transpiled on first use and cached. Build-time is the default for shipped code; load-time exists for development and for the migration window (§4).

3.4 No CGO, one artifact

Both `goja` and the `esbuild` Go API are pure Go. The binary therefore links statically with `CGO_ENABLED=0` and cross-compiles to every target with the standard Go toolchain—no native toolchain per architecture, no QEMU emulation

step. The same source produces an `amd64` and an `arm64` artifact in parallel on native runners. One artifact, parameterized by configuration, is the unit of deployment for every brand and tenant.

4 Migration Path

The framework’s purpose is to make migration *incremental* and *contract-preserving*. An existing TypeScript service ports in three stages, and the HTTP/ZAP contract is invariant across all three.

Stage 0 — lift. The service’s handlers are transpiled to ES5 with esbuild and registered in `zaprpc.Registry` as goja-evaluated handlers. The service now runs inside the unified binary. No handler logic changed; the wire contract is identical. Inter-subsystem calls that previously crossed the network (e.g. a fetch to the identity service) are rewritten at the registry boundary to resolve in-process, which is the only change a lift requires and is purely additive at the seam.

Stage 1 — strangle. Hot or contract-critical handlers are rewritten as native Go and swapped in at the registry. Because the registry resolves a method name to whichever implementation is bound, a Go handler and a goja handler are interchangeable behind the same name. Rewriting proceeds one method at a time with no flag day; at every point the binary is shippable.

Stage 2 — retire. When the last method behind a service is native Go, the goja path and the transpiled bundle for that service are dropped. The esbuild dependency remains only where JavaScript handlers remain.

Case study: tRPC to ZAP. The `hanzoai/esign` service is mid-migration from tRPC [7] to ZAP. tRPC’s typed router maps cleanly onto the ZAP method surface: a tRPC procedure becomes a ZAP method, and the end-to-end type safety tRPC provides on the wire is preserved by ZAP’s schema. In the most recent round, all fourteen

of fourteen routers are served server-side through the ZAP surface (14/14), with the client SDK re-targeted from the tRPC link to the ZAP transport. The contract seen by the frontend did not change; the transport beneath it did. This is the strangle stage in practice: the routers were ported behind a stable method surface without a coordinated client/server cutover.

5 Multi-Tenancy

The same artifact serves `api.hanzo.ai`, `api.osage.cloud`, `api.lux.cloud`, `api.zoo.cloud`, and every white-label tenant. Two things are configuration, not code: the *brand* (logo, title, identity issuer, default domain), resolved per request from the hostname; and the *enabled subsystem set*, resolved per deployment. A tenant that needs identity and commerce but not inference mounts only those modules; the binary is the same, the module graph differs.

This is the natural payoff of the module system and the registry. Mounting a subsystem is registering its methods; not mounting it is not registering them. Brand resolution is a middleware that reads the hostname and binds a brand context that subsystems consult for issuer, theme, and policy. Per HIP-0106, brand isolation is enforced at the data layer—every query is scoped to the tenant resolved from the authenticated identity—so a single process serving many brands never crosses tenant boundaries. The white-label separation rules (no cross-brand logos, hostname-based brand detection) are realized as configuration consumed by one binary, not as forked deployments.

6 The Code-Execution Boundary

The most important design line in this work separates *in-process evaluation of trusted code* from *sandboxed execution of untrusted code*. Conflating them is the failure mode the boundary exists to prevent.

In-process goja: trusted, fast, same address space. goja runs JavaScript in the binary’s own address space with no isolation boundary. That is precisely why it is appropriate only for *trusted* code: platform handlers authored by the platform team, transpiled from the platform’s own TypeScript, and shipped in the binary. For this code, sharing the address space is the point—a goja handler calls a Go subsystem through the registry with no serialization, and the performance floor is a function call. goja is the right tool when the code is trusted and the call is on the hot path (HIP-0105, the in-process extension runtime).

hanzoai/runtime: untrusted, isolated, separate. User-supplied code, agent-generated code, and computer-use workloads are *untrusted* and must not share the control plane’s address space. They run in **hanzoai/runtime**, a distinct platform that isolates execution in containers and microVMs with hardware-virtualization boundaries in the lineage of Firecracker [1] and the isolate-per-tenant model of edge runtimes [12, 4]. **runtime** ships its own SDKs (Go, Python, TypeScript) and is out of scope for the control plane binary except as a downstream callee.

The rule. Code the platform authors and trusts, on the hot path, runs in-process via goja. Code the platform receives and cannot trust runs in **runtime** behind a virtualization boundary. The two never swap roles. A handler is never promoted from **runtime** into goja, and untrusted input is never evaluated in goja. Stating the rule sharply is the contribution; the mechanisms on each side are conventional.

7 Cost Model

We compare a representative decomposed control plane against the unified binary. The decomposed baseline is N services, each in its own container with a sidecar proxy, joined by a service mesh; the unified case is one container per replica. We avoid fabricated benchmarks and parameterize the model by published overheads;

the figures below are illustrative of the *shape* of the saving, not a measurement of a specific deployment.

Container and proxy footprint. The decomposed stack pays, per service, for a container runtime slice and a sidecar. With N services and R replicas, that is $N \cdot R$ application containers *plus* $N \cdot R$ sidecars; the unified binary pays for R containers and no sidecars. The sidecar CPU and memory reservation is a fixed tax per pod that the service-mesh literature measures in the hundreds of millicores and tens to low-hundreds of megabytes under load [15]; multiplied by $N \cdot R$ it is the dominant avoidable cost.

Per-call overhead. Let a user action touch h subsystems on its critical path. The decomposed stack pays h network hops, each with two proxy traversals and a serialize/deserialize pair; the unified binary pays h function calls and zero serializations. With per-hop mesh overhead measured in the low single-digit milliseconds [15], a four-subsystem action sheds several milliseconds of pure overhead and the CPU that produced it.

Operational surface. N pipelines, N rollouts, N dashboards, and a mesh control plane collapse to one pipeline, one rollout, and one process to observe. This is not a compute cost but it is a real and recurring engineering cost; the Death-StarBench and service-mesh studies both note operational complexity as a first-order property of microservice topologies [5, 15].

What the unified binary does *not* save. It does not reduce the work the application actually performs, and it does not help when subsystems must scale independently—if inference needs ten times the replicas of identity, co-locating them wastes the smaller subsystem’s headroom. The model assumes co-scaling, which holds for the control plane and fails for the inference data plane; the latter is deployed separately for exactly this reason.

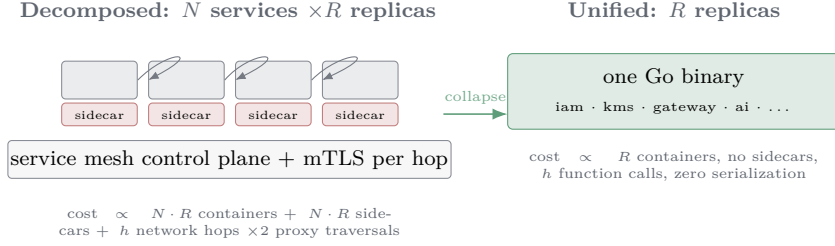


Figure 2: Illustrative cost comparison (shape, not measurement). The decomposed stack pays for $N \cdot R$ containers plus an equal number of sidecars and a mesh control plane; each user action crossing h subsystems pays h network hops with two proxy traversals each. The unified binary pays for R containers and turns the h hops into h function calls. Per-hop mesh overheads are taken from published service-mesh measurements [15]; the comparison assumes co-scaled subsystems (§7).

8 Performance

Function call vs. network RPC. The structural performance argument is that an in-process subsystem call is a Go function invocation—nanoseconds, no allocation beyond arguments, no serialization—against a network RPC’s microseconds-to-milliseconds of serialize, syscall, network, deserialize, and (with a mesh) two proxy traversals. The unified binary moves every co-located call from the latter class to the former. The latency floor of an intra-binary call is set by Go’s call and scheduling cost, not by the network stack.

Promise pipelining at the edge. For calls that *do* cross the network—external clients, peer clusters—ZAP’s promise pipelining [11] collapses dependent call chains into a single round trip. A client that creates a resource and immediately operates on it issues the second call against the promise of the first; the server resolves the chain locally. This bounds edge latency by one round trip per dependent chain rather than one per call, which is the same economy the in-process path gets for free, recovered for the cases that must remain remote.

goja overhead, honestly. A goja-evaluated handler is slower than the equivalent native Go handler: it interprets ECMAScript and bridges values across the Go/JS boundary. This is the cost of Stage 0 (§4) and the reason Stage 1 exists. The design does not pretend goja is free; it makes

the goja path a *temporary* state that the strangle stage retires on the hot path while keeping the cold path in JavaScript as long as that is cheaper to maintain than to rewrite.

9 Future Work

Three directions extend the framework without changing its shape. First, *additional in-process language runtimes*—a `pyvm` and a `v8vm` mounted behind the same registry seam—so trusted handlers in Python or via a V8 isolate dispatch identically to goja handlers, with the isolate option offering a stronger in-process boundary than goja for code that is trusted but warrants defense in depth. Second, *hot reload* of goja modules: because load-time transpilation already exists, swapping a handler bundle without restarting the process is a registry rebinding, useful in development and for cold-path handlers. Third, *gRPC-on-ZAP interop* [6], exposing the ZAP surface to gRPC clients so existing gRPC ecosystems consume the platform without a new SDK, and *deeper observability*, attributing power and CPU to in-process subsystems with the granularity that tools such as Kepler [2] bring to containers—harder inside one process, and the right place to invest the operational savings the unified binary returns.

10 Conclusion

For a co-located, co-released control plane, decomposition into networked microservices spends compute on proxies and serialization, adds latency in proportion to the hop count, and imposes a distributed-systems tax with no scaling benefit to justify it. The Hanzo unified cloud binary collapses the control plane into one Go process: `zip` over Fiber v3, ZAP serving both the wire and the in-process dispatch seam, goja and esbuild admitting TypeScript backends without a rewrite, and a strangle path to native Go that never breaks the contract. One artifact serves every brand and tenant by configuration. The boundary against untrusted code—in-process goja for trusted hot paths, the sandboxed `runtime` for everything else—is the load-bearing design decision, and we have stated it sharply. The saving is not magic; it is the removal of plumbing that the workload never needed.

References

- [1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight Virtualization for Serverless Applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434. USENIX Association, 2020.
- [2] Cloud Native Computing Foundation. Kefler: Kubernetes-based Efficient Power Level Exporter. <https://sustainable-computing.io>, 2024.
- [3] Fiber Authors. Fiber: An Express-inspired Web Framework for Go. <https://github.com/gofiber/fiber>, 2025.
- [4] Phani Kishore Gadepalli, Sean McBride, Gregor Peach, Ludmila Cherkasova, and Gabriel Parmer. Sledge: A Serverless-First, Lightweight Wasm Runtime for the Edge. In *Proceedings of the 21st International Middleware Conference*, pages 265–279. ACM, 2020.
- [5] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, et al. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 3–18. ACM, 2019.
- [6] gRPC Authors. gRPC: A High Performance, Open Source Universal RPC Framework. <https://grpc.io>, 2025.
- [7] Alex Johansson and tRPC contributors. tRPC: End-to-End Typesafe APIs Made Easy. <https://trpc.io>, 2025.
- [8] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2nd edition, 2021.
- [9] Dmitry Panov and goja contributors. goja: ECMAScript/JavaScript Engine in Pure Go. <https://github.com/dop251/goja>, 2025.
- [10] Prime Video Tech. Scaling up the Prime Video Audio/Video Monitoring Service and Reducing Costs by 90%. Amazon Prime Video Engineering Blog, 2023.
- [11] Kenton Varda. Cap’n Proto: Serialization and RPC with Promise Pipelining. <https://capnproto.org>, 2013.
- [12] Kenton Varda. Fine-Grained Sandboxing with V8 Isolates. Cloudflare Blog, 2018. <https://blog.cloudflare.com/cloud-computing-without-containers/>.
- [13] Steve Vinoski. Convenience over Correctness. *IEEE Internet Computing*, 12(4):89–92, 2008.
- [14] Evan Wallace. esbuild: An Extremely Fast Bundler for the Web. <https://github.com/evanw/esbuild>, 2025.
- [15] Xiangfeng Zhu, Guozhen She, Bowen Xue, Yu Zhang, Yongsu Zhang, Xuan Kelvin Zou, Xiongchun Duan, Peng He, Arvind Krishnamurthy, Matthew Lentz, Danyang Zhuo, and Ratul Mahajan. Dissecting Service Mesh Overheads. In *arXiv preprint arXiv:2207.00592*, 2022.